

AGV 算法可复用母模块

agv_algorithm_mother_modules_v2.py

源码行数：1051 行

排版方式：A4 横向、带行号、自动软换行、保留中文注释与缩进

颜色提示：绿色=注释，蓝色=class/def，紫色=import/装饰器

说明：软换行只影响纸面显示，不修改原始 .py 文件。

生成日期：2026-07-19

打印建议

A4、横向、双面、实际大小/100%，不要选择“每张多页”。

需要修改或运行时仍使用原始 .py 文件；PDF只用于阅读、批注和课堂打印。

```
1  """
2  AGV 智能调度算法可复用母模块 V2（单文件教学版）
3  =====
4
5  目标:
6  1. 用一套固定接口容纳规则、精确优化、元启发式、强化学习、多智能体和混合算法。
7  2. 提供一个不依赖第三方包、可以直接运行的基础组合:
8     规则派工 + Dijkstra 最短路 + 约束过滤 + 时空预约 + 安全校验 + 评估。
9  3. 把“环境、决策、经验、学习、路径、冲突、安全、评估”拆成可替换模块。
10
11  运行: python agv_algorithm_mother_modules_v2.py
12  测试: python -m unittest -v test_agv_algorithm_mother_modules_v2.py
13
14  说明: 这是一份母框架, 不是某一家工厂可以直接上线的 WCS/WES。
15  真实项目还要接入地图、PLC/调度接口、订单流、充电规则、故障处理和数据库。
16  """
17
18  from __future__ import annotations
19
20  from abc import ABC, abstractmethod
21  from collections import defaultdict, deque
22  from dataclasses import dataclass, field, replace
23  from heapq import heappop, heappush
24  import math
25  import random
26  from typing import Any, Callable, Hashable, Iterable, Sequence
27
28
29  # =====
30  # 第 1 层: 问题定义与状态 —— “要解决什么”
31  # =====
32
33
34  @dataclass(frozen=True)
35  class AGV:
36      """一台 AGV 在某个决策时刻的状态。"""
37
38      agv_id: str
39      node: str
40      battery: float
41      capacity: float
42      busy: bool = False
43      faulted: bool = False
44
45
46  @dataclass(frozen=True)
47  class Task:
48      """一个运输任务。priority 越大, 越希望优先处理。"""
49
50      task_id: str
51      pickup: str
52      dropoff: str
53      load: float
54      priority: float = 1.0
55      assigned: bool = False
56      due_time: int | None = None
57
```

```
58
59 @dataclass(frozen=True)
60 class SchedulingSnapshot:
61     """一次决策看到的完整快照；对应强化学习中的 observation/state。"""
62
63     time: int
64     agvs: tuple[AGV, ...]
65     tasks: tuple[Task, ...]
66     metadata: dict[str, Any] = field(default_factory=dict)
67
68     def get_agv(self, agv_id: str) -> AGV:
69         return next(x for x in self.agvs if x.agv_id == agv_id)
70
71     def get_task(self, task_id: str) -> Task:
72         return next(x for x in self.tasks if x.task_id == task_id)
73
74
75 class WeightedGraph:
76     """轻量有权图；可替换为真实路网、NetworkX 或数字孪生地图服务。"""
77
78     def __init__(self) -> None:
79         self._adj: dict[str, list[tuple[str, float]]] = defaultdict(list)
80
81     def add_edge(self, a: str, b: str, cost: float = 1.0, bidirectional: bool = True) -> None:
82         if cost <= 0:
83             raise ValueError("边权必须大于 0")
84         self._adj[a].append((b, float(cost)))
85         self._adj.setdefault(b, [])
86         if bidirectional:
87             self._adj[b].append((a, float(cost)))
88
89     def neighbors(self, node: str) -> tuple[tuple[str, float], ...]:
90         return tuple(self._adj.get(node, ()))
91
92     def shortest_path(
93         self,
94         start: str,
95         goal: str,
96         heuristic: Callable[[str, str], float] | None = None,
97     ) -> tuple[list[str], float]:
98         """Dijkstra; 传入可采纳 heuristic 时就是 A* 的母模板。"""
99         if start == goal:
100             return [start], 0.0
101         h = heuristic or (lambda _a, _b: 0.0)
102         frontier: list[tuple[float, float, str]] = [(h(start, goal), 0.0, start)]
103         previous: dict[str, str | None] = {start: None}
104         best_g: dict[str, float] = {start: 0.0}
105
106         while frontier:
107             _f, g, node = heappop(frontier)
108             if g != best_g.get(node):
109                 continue
110             if node == goal:
111                 path: list[str] = []
112                 cursor: str | None = goal
113                 while cursor is not None:
114                     path.append(cursor)
```

```

115         cursor = previous[cursor]
116         return list(reversed(path)), g
117     for nxt, edge_cost in self.neighbors(node):
118         new_g = g + edge_cost
119         if new_g < best_g.get(nxt, math.inf):
120             best_g[nxt] = new_g
121             previous[nxt] = node
122             heappush(frontier, (new_g + h(nxt, goal), new_g, nxt))
123     raise ValueError(f"节点 {start!r} 与 {goal!r} 之间不存在可行路径")
124
125
126 @dataclass(frozen=True)
127 class ProblemDefinition:
128     """项目级固定配置。新的 AGV 项目主要替换这里和状态构造器。"""
129
130     graph: WeightedGraph
131     min_battery: float = 15.0
132     energy_per_distance: float = 1.0
133     max_wait_steps: int = 50
134     weights: dict[str, float] = field(
135         default_factory=lambda: {
136             "travel": 1.0,
137             "tardiness": 2.0,
138             "energy": 0.2,
139             "priority": 1.0,
140         }
141     )
142
143
144 class StateBuilder:
145     """把数据库/数字孪生/仿真器原始数据整理为统一快照。"""
146
147     def build(
148         self,
149         time: int,
150         agvs: Iterable[AGV],
151         tasks: Iterable[Task],
152         metadata: dict[str, Any] | None = None,
153     ) -> SchedulingSnapshot:
154         return SchedulingSnapshot(time, tuple(agvs), tuple(tasks), metadata or {})
155
156
157 class FeaturePipeline:
158     """
159     感知/预测/表征母模块。
160
161     YOLO、LSTM、Transformer、GNN 等通常只是流水线中的一个变换，
162     最终输出仍要交给 DecisionEngine，而不是直接绕过约束和安全层。
163     """
164
165     def __init__(self, transforms: Sequence[Callable[[Any], Any]]) -> None:
166         self.transforms = tuple(transforms)
167
168     def transform(self, raw_input: Any) -> Any:
169         value = raw_input
170         for transform in self.transforms:
171             value = transform(value)

```

```

172         return value
173
174
175 # =====
176 # 第 2 层: 候选、约束与目标 —— “哪些能做、哪个好”
177 # =====
178
179
180 @dataclass(frozen=True)
181 class CandidateDecision:
182     agv_id: str
183     task_id: str
184     score: float = math.inf
185     explanation: str = ""
186
187
188 class CandidateGenerator:
189     """生成 AGV × 任务候选对; 复杂项目可加入候选任务窗口或邻域剪枝。"""
190
191     def generate(self, snapshot: SchedulingSnapshot) -> List[CandidateDecision]:
192         agvs = [a for a in snapshot.agvs if not a.busy and not a.faulted]
193         tasks = [t for t in snapshot.tasks if not t.assigned]
194         return [CandidateDecision(a.agv_id, t.task_id) for a in agvs for t in tasks]
195
196
197 class ConstraintChecker:
198     """硬约束过滤器: 违反即不能执行, 不应只靠负奖励碰运气。"""
199
200     def is_feasible(
201         self,
202         problem: ProblemDefinition,
203         snapshot: SchedulingSnapshot,
204         candidate: CandidateDecision,
205     ) -> tuple[bool, str]:
206         agv = snapshot.get_agv(candidate.agv_id)
207         task = snapshot.get_task(candidate.task_id)
208         if agv.busy or agv.faulted:
209             return False, "AGV 忙碌或故障"
210         if task.assigned:
211             return False, "任务已分配"
212         if task.load > agv.capacity:
213             return False, "载荷超过容量"
214         try:
215             _, to_pickup = problem.graph.shortest_path(agv.node, task.pickup)
216             _, to_dropoff = problem.graph.shortest_path(task.pickup, task.dropoff)
217         except ValueError:
218             return False, "路网不连通"
219         predicted_battery = agv.battery - (to_pickup + to_dropoff) * problem.energy_per_distance
220         if predicted_battery < problem.min_battery:
221             return False, "预计执行后电量低于安全阈值"
222         return True, "可行"
223
224     def filter(
225         self,
226         problem: ProblemDefinition,
227         snapshot: SchedulingSnapshot,
228         candidates: Iterable[CandidateDecision],

```

```

229     ) -> list[CandidateDecision]:
230         return [c for c in candidates if self.is_feasible(problem, snapshot, c)[0]]
231
232
233     @dataclass(frozen=True)
234     class ActionMask:
235         """把硬约束转换成策略能使用的合法动作掩码。"""
236
237         allowed: tuple[bool, ...]
238
239         @classmethod
240         def from_keys(
241             cls,
242             all_action_keys: Sequence[Hashable],
243             feasible_action_keys: set[Hashable],
244         ) -> "ActionMask":
245             return cls(tuple(key in feasible_action_keys for key in all_action_keys))
246
247         @classmethod
248         def from_candidates(
249             cls,
250             all_candidates: Sequence[CandidateDecision],
251             feasible_candidates: Sequence[CandidateDecision],
252         ) -> "ActionMask":
253             feasible = {(x.agv_id, x.task_id) for x in feasible_candidates}
254             keys = [(x.agv_id, x.task_id) for x in all_candidates]
255             return cls.from_keys(keys, feasible)
256
257         def legal_indices(self) -> list[int]:
258             return [i for i, allowed in enumerate(self.allowed) if allowed]
259
260         def validate(self, action_index: int) -> None:
261             if action_index < 0 or action_index >= len(self.allowed):
262                 raise ValueError(f"动作索引 {action_index} 越界")
263             if not self.allowed[action_index]:
264                 raise ValueError(f"动作索引 {action_index} 被硬约束屏蔽")
265
266
267     class ObjectiveEvaluator:
268         """统一目标函数：规则、优化和学习算法都用同一评价口径。"""
269
270         def score(
271             self,
272             problem: ProblemDefinition,
273             snapshot: SchedulingSnapshot,
274             candidate: CandidateDecision,
275         ) -> CandidateDecision:
276             agv = snapshot.get_agv(candidate.agv_id)
277             task = snapshot.get_task(candidate.task_id)
278             _, empty_distance = problem.graph.shortest_path(agv.node, task.pickup)
279             _, loaded_distance = problem.graph.shortest_path(task.pickup, task.dropoff)
280             travel = empty_distance + loaded_distance
281             energy = travel * problem.energy_per_distance
282             tardiness = 0.0
283             if task.due_time is not None:
284                 tardiness = max(0.0, snapshot.time + travel - task.due_time)
285             w = problem.weights

```

```

286         total = (
287             w.get("travel", 1.0) * travel
288             + w.get("energy", 0.0) * energy
289             + w.get("tardiness", 0.0) * tardiness
290             - w.get("priority", 0.0) * task.priority
291         )
292         reason = (
293             f"空驶={empty_distance:.1f}, 载货={loaded_distance:.1f}, "
294             f"能耗={energy:.1f}, 延误={tardiness:.1f}, 优先级={task.priority:.1f}"
295         )
296         return replace(candidate, score=total, explanation=reason)
297
298
299 class RiskEvaluator:
300     """随机/鲁棒/风险敏感优化共用的场景代价聚合器。"""
301
302     def aggregate(
303         self,
304         scenario_costs: Sequence[float],
305         mode: str = "mean",
306         alpha: float = 0.9,
307     ) -> float:
308         if not scenario_costs:
309             raise ValueError("scenario_costs 不能为空")
310         costs = sorted(float(x) for x in scenario_costs)
311         if mode == "mean":
312             return sum(costs) / len(costs)
313         if mode == "worst":
314             return costs[-1]
315         if mode == "cvar":
316             if not 0 <= alpha < 1:
317                 raise ValueError("CVaR alpha 必须位于 [0, 1)")
318             tail_size = max(1, math.ceil((1.0 - alpha) * len(costs)))
319             tail = costs[-tail_size:]
320             return sum(tail) / len(tail)
321         raise ValueError("mode 只支持 mean、worst 或 cvar")
322
323
324 class MultiObjectiveEvaluator:
325     """多目标优化的Pareto前沿母模块。"""
326
327     def __init__(self, minimize: Sequence[str] = (), maximize: Sequence[str] = ()) -> None:
328         if not minimize and not maximize:
329             raise ValueError("至少指定一个最小化或最大化目标")
330         self.minimize = tuple(minimize)
331         self.maximize = tuple(maximize)
332
333     def dominates(self, left: dict[str, Any], right: dict[str, Any]) -> bool:
334         no_worse = True
335         strictly_better = False
336         for name in self.minimize:
337             no_worse &= left[name] <= right[name]
338             strictly_better |= left[name] < right[name]
339         for name in self.maximize:
340             no_worse &= left[name] >= right[name]
341             strictly_better |= left[name] > right[name]
342         return bool(no_worse and strictly_better)

```

```

343
344     def pareto_front(self, records: Sequence[dict[str, Any]]) -> list[dict[str, Any]]:
345         return [
346             record
347             for i, record in enumerate(records)
348             if not any(
349                 i != j and self.dominates(other, record)
350                 for j, other in enumerate(records)
351             )
352         ]
353
354
355 # =====
356 # 第 3 层：决策引擎 —— “由哪类算法做选择”
357 # =====
358
359
360 class DecisionEngine(ABC):
361     """所有算法共同遵守的母接口。"""
362
363     name = "decision-engine"
364
365     def reset(self, problem: ProblemDefinition) -> None:
366         """新实验开始时清理内部状态；无状态算法可不做任何事。"""
367
368     @abstractmethod
369     def decide(
370         self,
371         problem: ProblemDefinition,
372         snapshot: SchedulingSnapshot,
373         candidates: Sequence[CandidateDecision],
374     ) -> CandidateDecision:
375         raise NotImplementedError
376
377     def update(self, feedback: Any) -> None:
378         """在线学习或滚动优化可接收反馈；静态规则可忽略。"""
379
380
381 class RuleDecisionEngine(DecisionEngine):
382     """可解释基线：计算所有候选分数并取最小值。"""
383
384     name = "weighted-nearest-rule"
385
386     def __init__(self, evaluator: ObjectiveEvaluator | None = None) -> None:
387         self.evaluator = evaluator or ObjectiveEvaluator()
388
389     def decide(self, problem, snapshot, candidates):
390         if not candidates:
391             raise RuntimeError("当前没有可行的 AGV—任务候选")
392         scored = [self.evaluator.score(problem, snapshot, c) for c in candidates]
393         return min(scored, key=lambda x: (x.score, x.agv_id, x.task_id))
394
395
396 class ExactSolverAdapter(DecisionEngine):
397     """
398     MILP/CP-SAT/网络流等精确优化的适配母类。
399

```



```

400     子类只需要把统一问题翻译给求解器，并将解翻译回 CandidateDecision。
401     真实项目可分别实现 OrToolsCPSATEngine、GurobiMILPEngine 等。
402     """
403
404     name = "exact-solver-adapter"
405
406     @abstractmethod
407     def build_model(self, problem, snapshot, candidates) -> Any:
408         raise NotImplementedError
409
410     @abstractmethod
411     def solve_model(self, model: Any) -> Any:
412         raise NotImplementedError
413
414     @abstractmethod
415     def decode_solution(self, raw_solution: Any) -> CandidateDecision:
416         raise NotImplementedError
417
418     def decide(self, problem, snapshot, candidates):
419         model = self.build_model(problem, snapshot, candidates)
420         return self.decode_solution(self.solve_model(model))
421
422
423 class MetaheuristicEngine:
424     """GA/PSO/ACO/VNS/ALNS 等可共用的“解—评价—扰动—接受”母循环。"""
425
426     def __init__(self, iterations: int = 100, seed: int = 0) -> None:
427         self.iterations = iterations
428         self.rng = random.Random(seed)
429
430     def optimize(
431         self,
432         initial_solution: Any,
433         objective: Callable[[Any], float],
434         neighbor: Callable[[Any, random.Random], Any],
435         repair: Callable[[Any], Any] | None = None,
436         accept: Callable[[float, float, int], bool] | None = None,
437     ) -> tuple[Any, float]:
438         repair_fn = repair or (lambda x: x)
439         accept_fn = accept or (lambda new, old, _i: new <= old)
440         current = repair_fn(initial_solution)
441         current_score = objective(current)
442         best, best_score = current, current_score
443         for i in range(self.iterations):
444             trial = repair_fn(neighbor(current, self.rng))
445             trial_score = objective(trial)
446             if accept_fn(trial_score, current_score, i):
447                 current, current_score = trial, trial_score
448             if trial_score < best_score:
449                 best, best_score = trial, trial_score
450         return best, best_score
451
452
453 class RLDecisionAdapter(DecisionEngine):
454     """
455     PPO/DQN/SAC 等模型无关强化学习的统一适配器。
456

```

```

457     encoder: SchedulingSnapshot -> 算法需要的 observation
458     predictor: observation, valid_action_ids -> action_id
459     decoder: action_id, candidates -> CandidateDecision
460     """
461
462     name = "rl-policy-adapter"
463
464     def __init__(
465         self,
466         encoder: Callable[[SchedulingSnapshot], Any],
467         predictor: Callable[[Any, List[int]], int],
468     ) -> None:
469         self.encoder = encoder
470         self.predictor = predictor
471
472     def decide(self, problem, snapshot, candidates):
473         if not candidates:
474             raise RuntimeError("没有可行动作")
475         observation = self.encoder(snapshot)
476         valid_action_ids = list(range(len(candidates))) # action mask 的最小形式
477         action_id = self.predictor(observation, valid_action_ids)
478         if action_id not in valid_action_ids:
479             raise RuntimeError(f"策略返回了非法动作 {action_id}")
480         return candidates[action_id]
481
482
483 class MARLDecisionAdapter(DecisionEngine):
484     """IQL/QMIX/MADDPG/MAPPO 等多智能体算法的联合动作适配母类。"""
485
486     name = "marl-adapter"
487
488     @abstractmethod
489     def decide_jointly(
490         self, problem: ProblemDefinition, snapshot: SchedulingSnapshot
491     ) -> dict[str, Any]:
492         raise NotImplementedError
493
494     def decide(self, problem, snapshot, candidates):
495         joint = self.decide_jointly(problem, snapshot)
496         agv_id = str(joint["agv_id"])
497         task_id = str(joint["task_id"])
498         match = next((c for c in candidates if c.agv_id == agv_id and c.task_id == task_id), None)
499         if match is None:
500             raise RuntimeError("多智能体联合动作不满足当前约束")
501         return match
502
503
504 class ModelBasedPlanner:
505     """
506     模型式强化学习/学习模型+MPC的通用短视域规划器。
507
508     transition_model 预测下一状态, reward_model 评价一步结果;
509     planner 在模型中向前滚动, 而不是只凭当前策略直接出动作。
510     """
511
512     def __init__(
513         self,

```

```

514     action_provider: Callable[[Any], Sequence[Any]],
515     transition_model: Callable[[Any, Any], Any],
516     reward_model: Callable[[Any, Any, Any], float],
517     horizon: int = 3,
518     discount: float = 0.99,
519     beam_width: int = 16,
520 ) -> None:
521     if horizon <= 0 or beam_width <= 0:
522         raise ValueError("horizon 和 beam_width 必须大于 0")
523     self.action_provider = action_provider
524     self.transition_model = transition_model
525     self.reward_model = reward_model
526     self.horizon = horizon
527     self.discount = discount
528     self.beam_width = beam_width
529
530 def plan(self, initial_state: Any) -> tuple[Any, float, list[Any]]:
531     beams: list[tuple[float, Any, list[Any]]] = [(0.0, initial_state, [])]
532     for depth in range(self.horizon):
533         expanded: list[tuple[float, Any, list[Any]]] = []
534         for score, state, sequence in beams:
535             for action in self.action_provider(state):
536                 next_state = self.transition_model(state, action)
537                 reward = self.reward_model(state, action, next_state)
538                 total = score + (self.discount**depth) * reward
539                 expanded.append((total, next_state, sequence + [action]))
540         if not expanded:
541             break
542         expanded.sort(key=lambda item: item[0], reverse=True)
543         beams = expanded[: self.beam_width]
544     if not beams or not beams[0][2]:
545         raise RuntimeError("模型规划器没有找到可行动作")
546     best_score, _state, sequence = beams[0]
547     return sequence[0], best_score, sequence
548
549
550 class HierarchicalController:
551     """分层强化学习/两阶段调度的最小母接口。"""
552
553     def __init__(
554         self,
555         high_level_policy: Callable[[Any], Any],
556         low_level_policy: Callable[[Any, Any], Any],
557     ) -> None:
558         self.high_level_policy = high_level_policy
559         self.low_level_policy = low_level_policy
560
561     def decide(self, state: Any) -> tuple[Any, Any]:
562         subgoal = self.high_level_policy(state)
563         primitive_action = self.low_level_policy(state, subgoal)
564         return subgoal, primitive_action
565
566
567 class GameCoordinator:
568     """势博弈/非合作博弈/拍卖前的通用最优响应母循环。"""
569
570     def __init__(

```

```

571     self,
572     players: Sequence[Hashable],
573     action_provider: Callable[[Hashable, Any], Sequence[Any]],
574     utility: Callable[[Hashable, Any, dict[Hashable, Any], Any], float],
575 ) -> None:
576     self.players = tuple(players)
577     self.action_provider = action_provider
578     self.utility = utility
579
580     def best_response(
581         self,
582         state: Any,
583         initial: dict[Hashable, Any],
584         rounds: int = 20,
585     ) -> dict[Hashable, Any]:
586         joint = dict(initial)
587         for _ in range(rounds):
588             changed = False
589             for player in self.players:
590                 actions = list(self.action_provider(player, state))
591                 if not actions:
592                     continue
593                 best = max(
594                     actions,
595                     key=lambda action: (
596                         self.utility(player, action, joint, state),
597                         str(action),
598                     ),
599                 )
600                 changed |= joint.get(player) != best
601                 joint[player] = best
602             if not changed:
603                 break
604         return joint
605
606
607 # =====
608 # 第 4 层: 路径、冲突与安全 —— “怎么安全执行”
609 # =====
610
611
612 class PathPlanner:
613     """路径规划统一入口; 可替换为 A*、SIPP、CBS、D* Lite 或外部服务。"""
614
615     def plan(self, graph: WeightedGraph, start: str, goal: str) -> tuple[list[str], float]:
616         return graph.shortest_path(start, goal)
617
618
619 class ReservationTable:
620     """简化时空预约表: 检查节点冲突和对向边冲突。"""
621
622     def __init__(self) -> None:
623         self.nodes: dict[tuple[int, str], str] = {}
624         self.edges: dict[tuple[int, str, str], str] = {}
625
626     def is_free(self, path: Sequence[str], start_time: int, owner: str = "candidate") -> bool:
627         for i, node in enumerate(path):

```

```
628         t = start_time + i
629         occupant = self.nodes.get((t, node))
630         if occupant is not None and occupant != owner:
631             return False
632         if i > 0:
633             prev = path[i - 1]
634             opposite = self.edges.get((t, node, prev))
635             if opposite is not None and opposite != owner:
636                 return False
637     return True
638
639     def reserve(self, path: Sequence[str], start_time: int, owner: str) -> None:
640         for i, node in enumerate(path):
641             t = start_time + i
642             self.nodes[(t, node)] = owner
643             if i > 0:
644                 self.edges[(t, path[i - 1], node)] = owner
645
646     def resolve(
647         self,
648         path: Sequence[str],
649         start_time: int,
650         max_wait_steps: int = 50,
651         owner: str = "candidate",
652     ) -> tuple[list[str], int]:
653         if not path:
654             raise ValueError("路径不能为空")
655         for waits in range(max_wait_steps + 1):
656             delayed = [path[0]] * waits + list(path)
657             if self.is_free(delayed, start_time, owner):
658                 return delayed, waits
659         raise RuntimeError("在最大等待步数内无法消解路径冲突")
660
661 class ConflictResolver(ReservationTable):
662     """预约表的工程封装；以后可由CBS、ECBS或PIBT实现同一职责。"""
663
664     def resolve_and_reserve(
665         self,
666         path: Sequence[str],
667         start_time: int,
668         owner: str,
669         max_wait_steps: int = 50,
670     ) -> tuple[list[str], int]:
671         repaired, waits = self.resolve(path, start_time, max_wait_steps, owner)
672         self.reserve(repaired, start_time, owner)
673         return repaired, waits
674
675 @dataclass(frozen=True)
676 class ExecutionPlan:
677     agv_id: str
678     task_id: str
679     to_pickup: list[str]
680     to_dropoff: list[str]
681     distance: float
682     wait_steps: int = 0
```

```

685     safe: bool = False
686     explanation: str = ""
687
688     @property
689     def full_path(self) -> list[str]:
690         if not self.to_pickup:
691             return list(self.to_dropoff)
692         return list(self.to_pickup) + list(self.to_dropoff[1:])
693
694
695 class SafetyShield:
696     """安全盾：最终动作执行前再做确定性校验。"""
697
698     def validate(
699         self, problem: ProblemDefinition, snapshot: SchedulingSnapshot, plan: ExecutionPlan
700     ) -> tuple[bool, str]:
701         agv = snapshot.get_agv(plan.agv_id)
702         task = snapshot.get_task(plan.task_id)
703         if agv.faulted or agv.busy:
704             return False, "AGV 状态不可执行"
705         if task.load > agv.capacity:
706             return False, "载荷超限"
707         remaining = agv.battery - plan.distance * problem.energy_per_distance
708         if remaining < problem.min_battery:
709             return False, "执行后电量越过安全线"
710         if plan.full_path[0] != agv.node or plan.full_path[-1] != task.dropoff:
711             return False, "路径起终点与任务不一致"
712         return True, "安全校验通过"
713
714
715 # =====
716 # 第 5 层：经验与学习 —— “算法从什么数据中更新”
717 # =====
718
719
720 @dataclass(frozen=True)
721 class Transition:
722     state: Any
723     action: Any
724     reward: float
725     next_state: Any
726     done: bool
727     info: dict[str, Any] = field(default_factory=dict)
728
729
730 class ExperienceModule(ABC):
731     @abstractmethod
732     def add(self, transition: Transition) -> None:
733         raise NotImplementedError
734
735     @abstractmethod
736     def __len__(self) -> int:
737         raise NotImplementedError
738
739
740 class ReplayBuffer(ExperienceModule):
741     """DQN/DDPG/TD3/SAC 常用的离策略经验回放。"""

```

```
742
743     def __init__(self, capacity: int, seed: int = 0) -> None:
744         if capacity <= 0:
745             raise ValueError("capacity 必须大于 0")
746         self._items: deque[Transition] = deque(maxlen=capacity)
747         self._rng = random.Random(seed)
748
749     def add(self, transition: Transition) -> None:
750         self._items.append(transition)
751
752     def sample(self, batch_size: int) -> list[Transition]:
753         if batch_size > len(self._items):
754             raise ValueError("样本数不足")
755         return self._rng.sample(list(self._items), batch_size)
756
757     def __len__(self) -> int:
758         return len(self._items)
759
760
761 class RolloutBuffer(ExperienceModule):
762     """PPO/A2C 常用的同策略轨迹缓存；训练一次后通常清空。"""
763
764     def __init__(self) -> None:
765         self._items: list[Transition] = []
766
767     def add(self, transition: Transition) -> None:
768         self._items.append(transition)
769
770     def consume(self) -> list[Transition]:
771         batch, self._items = self._items, []
772         return batch
773
774     def __len__(self) -> int:
775         return len(self._items)
776
777
778 class LearningModule(ABC):
779     @abstractmethod
780     def learn(self, experience: ExperienceModule) -> dict[str, float]:
781         raise NotImplementedError
782
783
784 class TabularQLearner:
785     """Q-learning 的最小可运行内核；用于理解，不是复杂 AGV 的最终算法。"""
786
787     def __init__(self, alpha: float = 0.1, gamma: float = 0.95, seed: int = 0) -> None:
788         self.alpha = alpha
789         self.gamma = gamma
790         self.q: dict[tuple[Hashable, Hashable], float] = defaultdict(float)
791         self.rng = random.Random(seed)
792
793     def get_q(self, state: Hashable, action: Hashable) -> float:
794         return self.q[(state, action)]
795
796     def set_q(self, state: Hashable, action: Hashable, value: float) -> None:
797         self.q[(state, action)] = float(value)
798
```

```

799     def choose_action(
800         self, state: Hashable, actions: Sequence[Hashable], epsilon: float
801     ) -> Hashable:
802         if not actions:
803             raise ValueError("actions 不能为空")
804         if self.rng.random() < epsilon: # 探索
805             return self.rng.choice(list(actions))
806         return max(actions, key=lambda a: (self.get_q(state, a), str(a))) # 利用
807
808     def update(
809         self,
810         state: Hashable,
811         action: Hashable,
812         reward: float,
813         next_state: Hashable,
814         next_actions: Sequence[Hashable],
815         done: bool = False,
816     ) -> float:
817         old_q = self.get_q(state, action)
818         future = 0.0 if done or not next_actions else max(self.get_q(next_state, a) for a in next_actions)
819         target = reward + self.gamma * future
820         td_error = target - old_q
821         self.set_q(state, action, old_q + self.alpha * td_error)
822         return td_error
823
824
825 # =====
826 # 第 6 层: 混合协调、执行与评估 —— “把模块串成工程闭环”
827 # =====
828
829
830 class HybridCoordinator:
831     """上层派工 + 下层路径 + 冲突处理 + 安全盾的可运行组合。"""
832
833     def __init__(
834         self,
835         decision_engine: DecisionEngine,
836         candidate_generator: CandidateGenerator | None = None,
837         constraint_checker: ConstraintChecker | None = None,
838         path_planner: PathPlanner | None = None,
839         conflict_resolver: ReservationTable | None = None,
840         safety_shield: SafetyShield | None = None,
841     ) -> None:
842         self.decision_engine = decision_engine
843         self.candidate_generator = candidate_generator or CandidateGenerator()
844         self.constraint_checker = constraint_checker or ConstraintChecker()
845         self.path_planner = path_planner or PathPlanner()
846         self.conflict_resolver = conflict_resolver or ConflictResolver()
847         self.safety_shield = safety_shield or SafetyShield()
848
849     def plan_one(self, problem: ProblemDefinition, snapshot: SchedulingSnapshot) -> ExecutionPlan:
850         raw = self.candidate_generator.generate(snapshot)
851         feasible = self.constraint_checker.filter(problem, snapshot, raw)
852         decision = self.decision_engine.decide(problem, snapshot, feasible)
853         agv = snapshot.get_agv(decision.agv_id)
854         task = snapshot.get_task(decision.task_id)
855

```



```

856     pickup_path, pickup_cost = self.path_planner.plan(problem.graph, agv.node, task.pickup)
857     dropoff_path, dropoff_cost = self.path_planner.plan(problem.graph, task.pickup, task.dropoff)
858     original_full = pickup_path + dropoff_path[1:]
859     resolved_full, waits = self.conflict_resolver.resolve(
860         original_full,
861         start_time=snapshot.time,
862         max_wait_steps=problem.max_wait_steps,
863         owner=agv.agv_id,
864     )
865     # 等待都加在起点, 因此可按取货节点位置重新切分。
866     pickup_end = max(i for i, node in enumerate(resolved_full) if node == task.pickup)
867     to_pickup = resolved_full[: pickup_end + 1]
868     to_dropoff = resolved_full[pickup_end:]
869     plan = ExecutionPlan(
870         agv_id=agv.agv_id,
871         task_id=task.task_id,
872         to_pickup=to_pickup,
873         to_dropoff=to_dropoff,
874         distance=pickup_cost + dropoff_cost,
875         wait_steps=waits,
876         explanation=decision.explanation,
877     )
878     safe, safety_reason = self.safety_shield.validate(problem, snapshot, plan)
879     if not safe:
880         raise RuntimeError(f"安全盾拒绝计划: {safety_reason}")
881     plan = replace(plan, safe=True, explanation=f"{plan.explanation}; {safety_reason}")
882     self.conflict_resolver.reserve(plan.full_path, snapshot.time, agv.agv_id)
883     return plan
884
885
886 class Executor:
887     """教学用状态推进器; 真实项目应替换为 WCS/仿真器/数字孪生接口。"""
888
889     def apply(
890         self, problem: ProblemDefinition, snapshot: SchedulingSnapshot, plan: ExecutionPlan
891     ) -> SchedulingSnapshot:
892         agvs = tuple(
893             replace(
894                 a,
895                 node=snapshot.get_task(plan.task_id).dropoff,
896                 battery=a.battery - plan.distance * problem.energy_per_distance,
897                 busy=False,
898             )
899             if a.agv_id == plan.agv_id
900             else a
901             for a in snapshot.agvs
902         )
903         tasks = tuple(t for t in snapshot.tasks if t.task_id != plan.task_id)
904         return SchedulingSnapshot(
905             time=snapshot.time + len(plan.full_path) - 1,
906             agvs=agvs,
907             tasks=tasks,
908             metadata=dict(snapshot.metadata),
909         )
910
911
912 class DigitalTwinAdapter(ABC):

```

```
913     """数字孪生不是调度算法；它负责读取实时状态并下发计划。"""
914
915     @abstractmethod
916     def read_snapshot(self) -> SchedulingSnapshot:
917         raise NotImplementedError
918
919     @abstractmethod
920     def dispatch(self, plan: ExecutionPlan) -> None:
921         raise NotImplementedError
922
923
924     @dataclass
925     class EvaluationMetrics:
926         completed_tasks: int = 0
927         total_distance: float = 0.0
928         total_wait_steps: int = 0
929         safety_rejections: int = 0
930         decision_count: int = 0
931
932     @property
933     def average_distance(self) -> float:
934         return self.total_distance / self.completed_tasks if self.completed_tasks else 0.0
935
936
937     class Evaluator:
938         def __init__(self) -> None:
939             self.metrics = EvaluationMetrics()
940
941         def record(self, plan: ExecutionPlan) -> None:
942             self.metrics.completed_tasks += 1
943             self.metrics.decision_count += 1
944             self.metrics.total_distance += plan.distance
945             self.metrics.total_wait_steps += plan.wait_steps
946
947
948     class ExperimentManager:
949         """固定随机种子、算法、场景和评价指标，避免只看单次 reward。"""
950
951         def __init__(self, coordinator: HybridCoordinator, evaluator: Evaluator | None = None) -> None:
952             self.coordinator = coordinator
953             self.evaluator = evaluator or Evaluator()
954
955         def run_until_empty(
956             self,
957             problem: ProblemDefinition,
958             snapshot: SchedulingSnapshot,
959             max_decisions: int = 100,
960         ) -> tuple[SchedulingSnapshot, EvaluationMetrics, List[ExecutionPlan]]:
961             executor = Executor()
962             plans: list[ExecutionPlan] = []
963             current = snapshot
964             while current.tasks and len(plans) < max_decisions:
965                 plan = self.coordinator.plan_one(problem, current)
966                 plans.append(plan)
967                 self.evaluator.record(plan)
968                 current = executor.apply(problem, current, plan)
969             return current, self.evaluator.metrics, plans
```

```

970
971
972 # =====
973 # 第 7 层: 算法目录 —— “选算法时先看场景, 不看热度”
974 # =====
975
976
977 @dataclass(frozen=True)
978 class AlgorithmFamily:
979     family: str
980     representatives: tuple[str, ...]
981     fixed_mother_loop: str
982     preferred_scenes: str
983     reusable_modules: tuple[str, ...]
984
985
986 def build_algorithm_catalog() -> tuple[AlgorithmFamily, ...]:
987     """与打印版 PDF 对应的算法家族索引。"""
988     return (
989         AlgorithmFamily("派工规则", ("最近车", "最早可用", "EDD", "SPT", "组合优先级"), "生成候选→过滤→打分→取最优", "实时性强、规则明确、基线与兜底", ("CandidateGenerator", "ConstraintChecker", "ObjectiveEva
990             luator")),
991         AlgorithmFamily("精确优化", ("MILP", "MINLP", "CP-SAT", "网络流", "Benders", "列生成"), "变量→目标→约束→求解→解码", "中小规模、要求最优性界或严谨约束", ("ProblemDefinition", "ExactSolverAdapter")),
992         AlgorithmFamily("不确定与多目标优化", ("随机规划", "鲁棒优化", "DRO", "CVar", "NSGA-II", "MOEA/D"), "场景/风险/目标建模→求解→Pareto选择", "到达随机、时长波动、能耗与效率权衡", ("ObjectiveEvaluator", "E
993             xperimentManager")),
994         AlgorithmFamily("单车路径规划", ("Dijkstra", "A*", "D* Lite", "LPA*", "JPS", "Theta*", "SIPP"), "开放集→扩展→代价更新→回溯", "静态/动态路网中的单车最短安全路径", ("WeightedGraph", "PathPlanner")),
995         AlgorithmFamily("多车冲突与 MAPF", ("预约表", "Cooperative A*", "WHCA*", "CBS", "ECBS", "PIBT", "M*"), "路径→检测冲突→加约束/优先级→重规划", "多 AGV 节点、边和死锁冲突", ("ReservationTable", "SafetySh
996             ield")),
997         AlgorithmFamily("元启发式", ("GA", "PSO", "ACO", "SA", "TS", "VNS", "ALNS", "FOA", "FPA"), "编码→评价→搜索算子→接受→终止", "组合爆炸、混合约束、需要较好可行解", ("MetaheuristicEngine", "ConstraintChe
998             cker")),
999         AlgorithmFamily("表格强化学习", ("Q-learning", "SARSA", "Expected SARSA"), "交互→TD目标→更新Q值→探索衰减", "小离散状态; 教学、小基线", ("TabularQLearner", "ExperienceModule")),
1000         AlgorithmFamily("深度价值学习", ("DQN", "Double DQN", "Dueling DQN", "PER", "Rainbow"), "回放采样→TD目标→网络更新→目标网络", "离散动作、状态较大", ("ReplayBuffer", "RLDecisionAdapter")),
1001         AlgorithmFamily("策略梯度/Actor-Critic", ("REINFORCE", "A2C", "PPO", "DDPG", "TD3", "SAC"), "采样→优势/回报→策略与价值更新", "PPO离散/连续均可; SAC/TD3偏连续控制", ("RolloutBuffer", "RLDecisionAdapter"
1002             )),
1003         AlgorithmFamily("模型式强化学习", ("Dyna-Q", "PETS", "MBPO", "Dreamer", "MuZero", "学习模型+MPC"), "学习转移模型→虚拟滚动→规划/策略更新", "真实试错昂贵、可利用仿真或数字孪生", ("DigitalTwinAdapter", "H
1004             ybridCoordinator")),
1005         AlgorithmFamily("多智能体强化学习", ("IQL", "VDN", "QMIX", "COMA", "MADDPG", "MAPPO", "MASAC"), "局部观测→联合交互→集中训练/分散执行", "多AGV协同与通信", ("MARLDecisionAdapter", "SafetyShield")),
1006         AlgorithmFamily("分层强化学习", ("Options", "MAXQ", "Feudal", "Manager-Worker"), "高层选子目标→低层执行→跨层奖励", "任务分解、派工与路径的多时间尺度", ("HybridCoordinator", "RLDecisionAdapter")),
1007         AlgorithmFamily("安全/约束/离线/模仿", ("Action Mask", "Shield", "CMDP", "CPO", "BC", "GAIL", "CQL", "IQL"), "约束/数据→安全学习→部署前验证", "不能在线冒险、已有历史日志", ("ConstraintChecker", "Safety
1008             Shield", "ReplayBuffer")),
1009         AlgorithmFamily("博弈/拍卖/市场", ("势博弈", "演化博弈", "拍卖", "合同网"), "参与者→效用→响应/竞价→均衡/分配", "分布式资源竞争和能量均衡", ("ObjectiveEvaluator", "MARLDecisionAdapter")),
1010         AlgorithmFamily("感知/预测/表征", ("YOLO", "CNN", "LSTM", "GRU", "Transformer", "GNN"), "原始数据→特征/预测→交给决策器", "行人、拥堵、图结构和时序预测", ("StateBuilder", "RLDecisionAdapter")),
1011     )
1012
1013 # =====
1014 # 第 8 层: 可直接运行的最小示例
1015 # =====
1016
1017
1018 def build_demo() -> tuple[ProblemDefinition, SchedulingSnapshot]:
1019     graph = WeightedGraph()
1020     graph.add_edge("S", "P1", 2)
1021     graph.add_edge("S", "P2", 4)
1022     graph.add_edge("P1", "D1", 3)
1023     graph.add_edge("P2", "D1", 2)
1024     graph.add_edge("D1", "CHARGE", 2)
1025     problem = ProblemDefinition(graph=graph)

```

```
1020     snapshot = StateBuilder().build(  
1021         time=0,  
1022         agvs=(AGV("AGV-01", "S", 90, 2), AGV("AGV-02", "P2", 65, 1)),  
1023         tasks=(  
1024             Task("T-urgent", "P1", "D1", 1, priority=5, due_time=8),  
1025             Task("T-normal", "P2", "D1", 1, priority=1, due_time=12),  
1026         ),  
1027     )  
1028     return problem, snapshot  
1029  
1030  
1031 def main() -> None:  
1032     problem, snapshot = build_demo()  
1033     coordinator = HybridCoordinator(RuleDecisionEngine())  
1034     manager = ExperimentManager(coordinator)  
1035     final_state, metrics, plans = manager.run_until_empty(problem, snapshot)  
1036  
1037     print("AGV 智能调度可复用母模块 V2: 运行示例")  
1038     print("-" * 58)  
1039     for i, plan in enumerate(plans, 1):  
1040         print(f"决策 {i}: {plan.agv_id} 执行 {plan.task_id}")  
1041         print(f"  路径: {' ' -> ' '.join(plan.full_path)}")  
1042         print(f"  距离: {plan.distance:.1f}; 等待: {plan.wait_steps}; 安全: {plan.safe}")  
1043         print(f"  原因: {plan.explanation}")  
1044     print("-" * 58)  
1045     print(f"完成任务: {metrics.completed_tasks}")  
1046     print(f"总距离: {metrics.total_distance:.1f}")  
1047     print(f"最终时刻: {final_state.time}")  
1048  
1049  
1050 if __name__ == "__main__":  
1051     main()
```